

DESAFIO DATABRICKS

Comunidades

Lakehouse com Arquitetura Medallion

■ 90 minutos

■ 5 tabelas

■ Nível Intermediário

■ Community Edition

■ Contexto do Desafio

Você foi contratado como **Engenheiro de Dados** da **TechVenda**, um e-commerce de produtos eletrônicos e móveis. A empresa possui dados espalhados em arquivos CSV (exportados do sistema legado) e precisa de uma plataforma moderna de dados para responder perguntas estratégicas de negócio.

Qual o faturamento total por mês em 2024?

Quais vendedores geraram mais receita?

Quais os produtos mais vendidos por categoria?

Qual a taxa de cancelamento de pedidos?

Sua missão: construir um Lakehouse completo com arquitetura Medallion no Databricks.

■ Arquitetura do Desafio



■ Pre-requisitos

- Conta no **Databricks Community Edition** criada e ativa (community.cloud.databricks.com)
- Cluster criado com Runtime 13.x LTS ou superior
- Os 5 arquivos CSV carregados no DBFS (veja instrucoes abaixo)

Como fazer upload dos CSVs:

No menu lateral: **Data** → **Add Data** → **Upload File**. Carregue cada arquivo para o caminho **dbfs:/FileStore/desafio/**. Para verificar, execute em um notebook:

```
display(dbutils.fs.ls('dbfs:/FileStore/desafio/'))
```

■ Dados Disponíveis — 5 Tabelas

Arquivo	Descricao	Registros
clientes.csv	Cadastro com cidade/estado/status	25
produtos.csv	Catalogo com categoria e preco	20
vendedores.csv	Equipe de vendas por regioao	8
pedidos.csv	Cabecalho com status e desconto	200
itens_pedido.csv	Itens com quantidade e preco	~509

Relacionamento entre as tabelas:

```
clientes <==> pedidos <==> vendedores
```

```
pedidos <==> itens_pedido <==> produtos
```

1

Camada Bronze

20 min | 20 pontos

Crie o notebook **01_bronze.py** que:

- Le os 5 arquivos CSV do DBFS
- Salva cada um como **Delta Table** no caminho delta/bronze/
- Nao aplica nenhuma transformacao (dados brutos)
- Registra o schema de cada tabela com printSchema()

Caminho esperado apos execucao:

```
dbfs:/delta/bronze/clientes/ dbfs:/delta/bronze/produtos/  
dbfs:/delta/bronze/vendedores/ dbfs:/delta/bronze/pedidos/  
dbfs:/delta/bronze/itens_pedido/
```

2

Camada Silver

25 min | 30 pontos

Crie o notebook **02_silver.py** que:

- Le as tabelas Delta da camada Bronze
- Converte colunas de data para **DateTime**
- Remove registros com status = 'inativo' (clientes, produtos, vendedores)
- Filtra pedidos com status_pedido diferente de 'cancelado'
- Faz o join entre pedidos, itens_pedido, clientes, produtos e vendedores
- Calcula: **valor_total_item = quantidade x preco_unitario x (1 - desconto)**
- Salva resultado em delta/silver/pedidos_enriquecidos

Dica para o calculo do valor:

```
valor_total_item = quantidade * preco_unitario * (1 - desconto)
```

3

Camada Gold

25 min | 30 pontos

Crie o notebook **03_gold.py** que gera **3 visoes analiticas** a partir da Silver:

Gold 1 — Faturamento Mensal

- Agrupar por **ano** e **mes** (extraídos da data_pedido)
- Calcular: total_pedidos e receita_total
- Ordenar por data crescente
- Salvar em **delta/gold/faturamento_mensal**

Gold 2 — Ranking de Vendedores

- Agrupar por **nome_vendedor** e **regiao**
- Calcular: total_pedidos, receita_gerada e ticket_medio
- Ordenar por receita_gerada decrescente
- Salvar em **delta/gold/ranking_vendedores**

Gold 3 — Top Produtos por Categoria

- Agrupar por **categoria** e **nome_produto**
- Calcular: quantidade_vendida e receita_produto
- Usar **Window Function (RANK)** para rankear dentro de cada categoria

- Filtrar apenas o **Top 3** por categoria
- Salvar em **delta/gold/top_produtos_categoria**

Dica para Window Function:

```
from pyspark.sql.window import Window window_spec =  
Window.partitionBy('categoria').orderBy(F.desc('receita_produto')) df =  
df.withColumn('ranking', F.rank().over(window_spec))
```

4

Workflow — Orquestracao

15 min | 15 pontos

Crie um **Databricks Workflow** que execute os notebooks na ordem correta:

```
bronze ■■> silver ■■> gold
```

- Os 3 notebooks devem ser tarefas (tasks) separadas no Workflow
- Silver depende de Bronze | Gold depende de Silver
- Execute o Workflow e mostre todas as tasks com status verde (sucesso)

Como criar: Menu lateral > **Workflows** > **Create Job**. Adicione as 3 tasks e configure as dependencias no campo '**Depends on**'.

■ Criterios de Avaliacao

Tarefa	Criterio	Pts
Bronze	5 tabelas Delta criadas e registradas no schema	20
Silver	Joins corretos, tipagem e calculo com desconto	30
Gold	3 visoes analiticas com Window Function correta	30
Workflow	3 tasks com dependencias corretas e todas verdes	15
Codigo	Organizado, comentado e com nomes claros	5
TOTAL		100

■ Dicas Essenciais

Ler CSV no Databricks:

```
df = spark.read.format("csv").option("header","true").option("inferSchema","true").load("dbfs:/FileStore/desafio/clientes.csv")
```

Salvar como Delta:

```
df.write.format("delta").mode("overwrite").save("dbfs:/delta/bronze/clientes")
```

Registrar no metastore (para usar via SQL):

```
spark.sql("CREATE TABLE IF NOT EXISTS bronze.clientes USING DELTA LOCATION  
'dbfs:/delta/bronze/clientes'")
```

■ O que Entregar no Grupo

- **Print** do Workflow com as 3 tasks verdes (sucesso)
- **Print** de um display() de cada tabela Gold com os dados
- **Link** do GitHub com os notebooks (bonus — vale ponto extra!)

Poste tudo no grupo da Comunidades e marque como #desafio-databricks